

Formas de mockar jsonwebtoken com Vitest

Sem vitest-mock-extended

#	Utilitário	Mecanismo Vitest	Acesso ao mock
1	—	vi.mock + factory	vi.mocked()
2	—	vi.mock auto-mock	vi.mocked()
3	—	vi.spyOn	variável local

Com vitest-mock-extended

#	Utilitário VME	Mecanismo Vitest	Acesso ao mock
4	mock<typeof jwt>()	vi.mock factory	vi.mocked()
5	MockProxy<typeof jwt> (tipo)	auto-mock	cast direto
6	mock<typeof jwt>()	vi.doMock	variável externa direta

Observações gerais

- **1, 2, 4, 5** mockam o módulo inteiro; **3** atua apenas no método.
- **1, 2, 3** dependem de `as any` para satisfazer o compilador em TypeScript estrito.
- **4, 5, 6** eliminam `as any` com tipagem derivada de `typeof jwt`.
- **4 e 6** usam o mesmo utilitário VME, diferindo apenas no mecanismo de registro do mock.
- **5** usa `vitest-mock-extended` exclusivamente para tipagem, sem criar proxy próprio.
- **6** é a única que exige reimportação dinâmica (`await import(...)`).

Conceitos fundamentais

Antes dos exemplos, é necessário compreender o que cada mecanismo realmente faz — pois seus efeitos operam em camadas distintas: sistema de módulos, runtime e sistema de tipos.

vi.mock

Intercepta o sistema de módulos do Node **antes da execução** de qualquer código do arquivo de teste. Isso ocorre porque o Vitest aplica um processo de *hoisting*: independentemente de onde `vi.mock` esteja escrito no arquivo, ele é movido para o topo e executado antes de qualquer `import`.

O efeito prático é que **todo código que importar o módulo mockado — incluindo o SUT — receberá a versão substituída**, não o módulo real.

Há duas variantes:

Auto-mock — sem factory: o Vitest analisa a forma real do módulo e substitui todas as exportações por `vi.fn()`. A estrutura é inferida, mas a tipagem resultante é genérica.

```
vi.mock('jsonwebtoken')
// jwt.verify agora é vi.fn(), mas TypeScript não sabe disso com precisão
```

Com factory — o segundo argumento define explicitamente o que o módulo exporta. Permite usar qualquer objeto, incluindo proxies do `vitest-mock-extended`:

```
vi.mock('jsonwebtoken', () => ({
  default: mock<typeof jwt>(),
}))
// a estrutura e a tipagem do mock são definidas pelo desenvolvedor
```

Restrição crítica do hoisting: por ser elevado antes dos `import`, a factory de `vi.mock` não pode referenciar variáveis declaradas no escopo do arquivo — elas ainda não existem no momento em que a factory é executada. Essa é a razão de existência do `vi.doMock`.

vi.doMock

Funcionalmente equivalente ao `vi.mock`, mas **sem hoisting**. É executado exatamente onde está escrito, em ordem com o restante do código.

Isso resolve a restrição do `vi.mock`, permitindo que a factory referencie variáveis do escopo externo:

```
const jwtMock = mock<typeof jwt>() // declarada antes

vi.doMock('jsonwebtoken', () => ({
  default: jwtMock, // possível apenas sem hoisting
}))
```

A consequência direta é que **módulos já importados estaticamente no topo do arquivo não são afetados** — eles foram resolvidos antes do `vi.doMock` ser executado. Por isso, todo módulo que deve receber a versão mockada precisa ser reimportado dinamicamente após o registro:

```
const { JwtAdapter } = await import('./jwt-adapter')
// agora JwtAdapter importará o jsonwebtoken mockado
```

Em síntese: `vi.doMock` troca a conveniência do hoisting automático pelo controle explícito do ciclo de vida do mock.

vi.mocked

Não cria mock algum. É exclusivamente uma **função de cast do sistema de tipos** — seu efeito existe apenas em tempo de compilação e desaparece completamente em runtime.

Seu propósito é informar ao TypeScript que uma importação que ele ainda trata como o tipo original (`typeof jwt`) é na verdade um mock (`MockedObject<typeof jwt>`), desbloqueando o acesso tipado a métodos como `mockReturnValueOnce` , `mockResolvedValueOnce` , `toHaveBeenCalledWith` etc.

```
vi.mocked(jwt.verify).mockReturnValueOnce({ sub: 'user_id' })
//                ^^^^^^^^^^^^^^^^^^^^^
// sem vi.mocked, TypeScript rejeita: jwt.verify não tem mockReturnValueOnce
// com vi.mocked, o compilador aceita pois sabe que é um MockedFunction
```

O objeto que `vi.mocked` recebe **já deve ser um mock em runtime** — criado por `vi.mock` , auto-mock ou `vitest-mock-extended` . `vi.mocked` apenas alinha o que o TypeScript enxerga com o que realmente existe em execução.

MockProxy<T> — vitest-mock-extended

É o **tipo TypeScript** que `vitest-mock-extended` atribui aos objetos criados por `mock<T>()` . Representa um objeto onde cada propriedade e método de `T` foi substituído por um `MockInstance` — o tipo do Vitest para funções mock.

Quando usado diretamente como cast (Forma 5), permite obter tipagem precisa sobre um mock criado pelo Vitest (auto-mock), sem precisar criar um proxy próprio:

```
const jwtMocked = jwt as unknown as MockProxy<typeof jwt>
// Em runtime: jwt é o auto-mock criado pelo Vitest (vi.fn() em cada método)
// Em compilação: TypeScript trata jwt como MockProxy, liberando mockReturnValueOnce etc.
```

O duplo cast `as unknown as MockProxy<T>` é necessário porque `typeof jwt` e `MockProxy<typeof jwt>` são tipos incompatíveis do ponto de vista do compilador — `unknown` serve como intermediário para forçar a conversão.

A diferença entre `MockProxy<T>` e `DeepMockProxy<T>` é de profundidade: `MockProxy` tipa apenas o primeiro nível do objeto; `DeepMockProxy` percorre recursivamente toda a estrutura de tipos, sendo necessário apenas quando há objetos aninhados com métodos próprios.

Relação entre os mecanismos

```
vi.mock / vi.doMock    → operam no sistema de módulos (runtime)
vi.mocked              → opera no sistema de tipos (compilação)
MockProxy<T>           → opera no sistema de tipos (compilação)
mock<T>() / mockDeep   → operam em ambos: criam proxy em runtime e carregam o tipo
```

Compreender essa separação é essencial para diagnosticar erros: se `mockReturnValueOnce` não existe em runtime, o problema está no mecanismo de módulo (`vi.mock / vi.doMock`). Se o TypeScript rejeita a chamada, o problema está na tipagem (`vi.mocked` ausente ou cast incorreto).

Exemplos

1 — vi.mock + factory function

```
import { describe, it, expect, vi, beforeEach } from 'vitest'
import jwt from 'jsonwebtoken'
import { JwtAdapter } from './jwt-adapter'

vi.mock('jsonwebtoken', () => ({
  default: {
    verify: vi.fn(),
    sign: vi.fn(),
  },
}))

describe('JwtAdapter', () => {
  let sut: JwtAdapter

  beforeEach(() => {
    sut = new JwtAdapter('any_secret')
    vi.clearAllMocks()
  })

  it('should return true when given a valid and non-expired token', async () => {
    // Arrange
    // "as any" necessário: vi.fn() não carrega a assinatura de jwt.verify
    vi.mocked(jwt.verify).mockReturnValueOnce({} as any)

    // Act
    const result = await sut.validate('valid_token')

    // Assert
    expect(result).toBe(true)
    expect(jwt.verify).toHaveBeenCalledWith('valid_token', 'any_secret')
  })
})
```

2 — vi.mock auto-mock + vi.mocked

```
import { describe, it, expect, vi, beforeEach } from 'vitest'
import jwt from 'jsonwebtoken'
import { JwtAdapter } from './jwt-adapter'

// Vitest infere a forma do módulo e substitui tudo por vi.fn()
vi.mock('jsonwebtoken')

describe('JwtAdapter', () => {
  let sut: JwtAdapter

  beforeEach(() => {
    sut = new JwtAdapter('any_secret')
    vi.clearAllMocks()
  })

  it('should return true when given a valid and non-expired token', async () => {
    // Arrange
    // "as any" necessário: tipo inferido pelo auto-mock não é preciso
    vi.mocked(jwt.verify).mockReturnValueOnce({} as any)

    // Act
    const result = await sut.validate('valid_token')

    // Assert
    expect(result).toBe(true)
    expect(jwt.verify).toHaveBeenCalledWith('valid_token', 'any_secret')
  })
})
```

3 — vi.spyOn

```
import { describe, it, expect, vi, afterEach } from 'vitest'
import jwt from 'jsonwebtoken'
import { JwtAdapter } from './jwt-adapter'

describe('JwtAdapter', () => {
  afterEach(() => {
    vi.restoreAllMocks()
  })

  it('should return true when given a valid and non-expired token', async () => {
    // Arrange
    // "as any" necessário: mockReturnValueOnce não infere o retorno de verify
    const verifySpy = vi.spyOn(jwt, 'verify').mockReturnValueOnce({} as any)

    const sut = new JwtAdapter('any_secret')

    // Act
    const result = await sut.validate('valid_token')

    // Assert
    expect(result).toBe(true)
    expect(verifySpy).toHaveBeenCalledWith('valid_token', 'any_secret')
  })
})
```

4 — mock<typeof jwt>() via vi.mock factory

```
import { describe, it, expect, vi, beforeEach } from 'vitest'
import { mock } from 'vitest-mock-extended'
import jwt from 'jsonwebtoken'
import { JwtAdapter } from './jwt-adapter'

vi.mock('jsonwebtoken', () => ({
  default: mock<typeof jwt>(),
}))

describe('JwtAdapter', () => {
  const jwtMocked = vi.mocked(jwt)

  let sut: JwtAdapter

  beforeEach(() => {
    sut = new JwtAdapter('any_secret')
    vi.clearAllMocks()
  })

  it('should return true when given a valid and non-expired token', async () => {
    // Arrange
    // Sem "as any": tipo inferido diretamente de typeof jwt.verify
    jwtMocked.verify.mockReturnValueOnce({ sub: 'user_id' })

    // Act
    const result = await sut.validate('valid_token')

    // Assert
    expect(result).toBe(true)
    expect(jwtMocked.verify).toHaveBeenCalledWith('valid_token', 'any_secret')
  })
})
```

5 — auto-mock + cast MockProxy<typeof jwt>

```
import { describe, it, expect, vi, beforeEach } from 'vitest'
import { type MockProxy } from 'vitest-mock-extended'
import jwt from 'jsonwebtoken'
import { JwtAdapter } from './jwt-adapter'

vi.mock('jsonwebtoken')

describe('JwtAdapter', () => {
  // O mock em runtime é criado pelo Vitest (auto-mock).
  // MockProxy contribui apenas com a tipagem – sem criar proxy próprio.
  const jwtMocked = jwt as unknown as MockProxy<typeof jwt>

  let sut: JwtAdapter

  beforeEach(() => {
    sut = new JwtAdapter('any_secret')
    vi.clearAllMocks()
  })

  it('should return true when given a valid and non-expired token', async () => {
    // Arrange
    // Sem "as any": MockProxy expõe mockReturnValueOnce com tipo correto
    jwtMocked.verify.mockReturnValueOnce({ sub: 'user_id' })

    // Act
    const result = await sut.validate('valid_token')

    // Assert
    expect(result).toBe(true)
    expect(jwtMocked.verify).toHaveBeenCalledWith('valid_token', 'any_secret')
  })
})
```

6 — mock<typeof jwt>() via vi.doMock

```
import { describe, it, expect, vi, beforeEach } from 'vitest'
import { mock } from 'vitest-mock-extended'
import type jwt from 'jsonwebtoken'
import { JwtAdapter } from './jwt-adapter'

// Variável acessível diretamente nos testes — possível apenas porque
// vi.doMock não é hoisted, ao contrário de vi.mock
const jwtMock = mock<typeof jwt>()

vi.doMock('jsonwebtoken', () => ({
  default: jwtMock,
}))

describe('JwtAdapter', () => {
  let sut: JwtAdapter

  beforeEach(async () => {
    vi.clearAllMocks()

    // Reimportação dinâmica obrigatória: garante que JwtAdapter
    // receba a versão mockada do módulo registrada pelo vi.doMock
    const { JwtAdapter: JwtAdapterFresh } = await import('./jwt-adapter')
    sut = new JwtAdapterFresh('any_secret')
  })

  it('should return true when given a valid and non-expired token', async () => {
    // Arrange
    // Sem "as any", sem vi.mocked(), sem cast: acesso direto à variável
    jwtMock.verify.mockReturnValueOnce({ sub: 'user_id' })

    // Act
    const result = await sut.validate('valid_token')

    // Assert
    expect(result).toBe(true)
    expect(jwtMock.verify).toHaveBeenCalledWith('valid_token', 'any_secret')
  })
})
```